

PROJECT REPORT

Robot Path Planning Simulation Using A* Algorithm

Submitted by:

**[Titiksha S,
Balambika V]**

[231BCADA17,
231BCADA21]

Submitted to:

[Sanjay Dutta]

[Advanced Computing]
[St Joseph's University]

SUMMARY

This project focused on testing the A* algorithm for robot path planning. I created a synthetic dataset with 100 different scenarios where a robot had to find the shortest path through a 20x20 grid filled with obstacles. The dataset had scenarios with obstacle densities ranging from 10% to 30%. I implemented the A* algorithm in Python using Google Colab and tested it on all 100 scenarios. The results showed that A* successfully found paths in 99 out of 100 cases. The average path cost was 24.85 units and the computation time was only 0.54 milliseconds. The algorithm was very efficient, exploring only 22% of available cells on average. This proves that A* is a fast and reliable algorithm for robot navigation.

INTRODUCTION

Robot path planning is an important problem in robotics and artificial intelligence. It means finding the best and optimal path for a robot to move from a starting point to a goal without hitting any obstacles. This is needed in many real-world applications like warehouse robots, self-driving cars, and delivery drones.

There are many algorithms for path planning. Some popular ones are Dijkstra's algorithm, Breadth-First Search, Depth-First Search and A*. Dijkstra's algorithm finds the shortest path but it is slow because it explores in all directions. A* is smarter because it uses a heuristic function to guide the search towards the goal. This makes it much faster.

The A* algorithm works by calculating the formula $f(n) = g(n) + h(n)$ for each cell. Where, $g(n)$ is the actual cost to reach that cell from the start, and $h(n)$ is the estimated cost from that cell to the goal. The algorithm always expands the cell with the lowest $f(n)$ value. If the heuristic never overestimates the actual cost (It should be admissible) A* is guaranteed to find the shortest path.

The objective of my project was to implement A* with a Euclidean distance heuristic, test it on 100 different scenarios, and analyse how it performs when obstacle density increases. I wanted to see if A* can handle different environments and still find good paths quickly.

LITERATURE REVIEW

Many researchers have worked on robot path planning algorithms.

[1] Miao and Tian (2013) proposed using Simulated Annealing for dynamic environments where obstacles move. Their method was flexible but could not guarantee finding the best path.

[2] Zhu et al. (2006) combined Artificial Potential Fields with Simulated Annealing to avoid getting stuck in local minima.

[3] Zhang et al. (2018) reviewed different path planning methods and found that no single algorithm works best in all situations. They recommended using hybrid approaches that combine global planning with local adjustments.

[4] Wang et al. (2011) tested Dijkstra's algorithm and showed that while it finds the optimal path, it is very slow because it explores too many nodes.

[5] Martinez-Alfaro and Gomez-Garcia (1998) used Simulated Annealing with fuzzy logic for path planning. Their approach worked well but was not reproducible because Simulated Annealing gives different results each time.

[6] Tsuzuki et al. (2006) also used Simulated Annealing by encoding paths as waypoints, but this method does not guarantee optimality.

[7] Janabi-Sharifi, F., & Vinke, D. (1993) used combined Artificial Potential Fields with Simulated Annealing to solve the local minima problem that happens in regular APF methods, but it took more computation time compared to using APF alone.

[8] Chen et al. (2009) studied path planning for industrial painting robots and emphasized that smooth collision-free paths are essential for quality.

[9] Bedaka and Lin (2018) developed a CAD-based simulation platform for testing path planning algorithms.

[10] Tavares et al. (2011) improved Simulated Annealing with adaptive neighborhood search.

From the literature, I learned that A* is widely recognized as a best algorithm because it guarantees optimal paths when using an admissible heuristic. Unlike probabilistic methods like Simulated Annealing, A* always gives the same result for the same input. This makes it reliable for practical applications.

METHODOLOGY

I followed these steps to complete my project:

Step 1: Dataset Generation - I wrote a Python script in Google Colab to generate a synthetic dataset. The dataset contained 100 scenarios. Each scenario had a 20x20 grid with randomly placed obstacles. I tested 5 different obstacle densities: 10%, 15%, 20%, 25%, and 30%. For each density level, I created 20 scenarios with different start and goal positions. I used a fixed random seed (42) so the results could be reproduced. The dataset was saved as an Excel file with columns for scenario ID, start position, goal position, obstacle rate, obstacle count, and obstacle coordinates in JSON format.

Step 2: A* Implementation - I implemented the A* algorithm from scratch in Python. The robot could move in 8 directions: up, down, left, right, and 4 diagonals. I used the Euclidean distance formula as the heuristic: $h(n) = \sqrt{(x_{\text{goal}} - x_{\text{current}})^2 + (y_{\text{goal}} - y_{\text{current}})^2}$. Straight moves had a cost of 1.0 and diagonal moves had a cost of $\sqrt{2} \approx 1.414$ because they cover more distance. The algorithm used a priority queue to always expand the most promising cell first.

Step 3: Running the Simulation - I loaded the dataset in Google Colab and ran the A* algorithm on each of the 100 scenarios. For each scenario, I recorded the path cost, number of nodes explored, computation time, and whether a path was found. I also calculated path efficiency as $(\text{straight line distance} / \text{actual path cost}) \times 100\%$.

RESULTS AND DISCUSSION

The A* algorithm successfully found paths in 99 out of 100 scenarios. Only one scenario at 30% obstacle density failed because the grid was completely blocked with no possible path. This is not a failure of the algorithm but rather a physically impossible situation. At 10% obstacle density, the average path cost was 24.44 units. As obstacles increased to 30%, the average path cost only went up to 25.42 units. This is only a 4% increase even though the obstacles tripled. This shows that A* is very good at finding alternate routes around obstacles. The number of nodes explored increased more significantly. At 10% obstacles, the algorithm explored an average of 68.6 nodes. At 30% obstacles, it explored 75.8 nodes. This makes sense because with more obstacles, the algorithm has to search more to find diversion. However, on average across all scenarios, only 22.32% of the free cells were explored. This proves that the Euclidean heuristic effectively guides the search.

The computation time was very stable across all obstacle densities, ranging from 0.52 to 0.55 milliseconds. This confirms that A* is fast enough for real-time applications. The path efficiency averaged 94.29%, meaning the paths found were very close to the theoretical straight-line distance.

When I compared my results with the literature review, A* performed much better than methods like Simulated Annealing. Unlike those probabilistic methods, A* gave consistent results every time and always found the optimal path when one existed.

CONCLUSION

My project demonstrated that the A* algorithm is a best choice for robot path planning in 2D grid environments. Out of 100 test scenarios, A* found optimal paths in 99 cases with an average computation time of only 0.54 milliseconds. The algorithm handled various obstacle densities correctly, with path cost increasing by only 4% when obstacles tripled from 10% to 30%. The main strength of A* is its use of a heuristic to guide the search efficiently. By exploring only 22% of available cells on average, it saves a lot of computation compared to algorithms like Dijkstra that explore everywhere. The Euclidean distance heuristic worked very well and guaranteed optimal paths.

For future work, this project could be extended to dynamic environments where obstacles move, or to 3D spaces instead of 2D grids.

REFERENCES

- [1] Miao, H., & Tian, Y. C. (2013). Dynamic robot path planning using an enhanced simulated annealing approach. *Applied Mathematics and Computation*, 222, 420–437.
- [2] Zhu, Q., Yan, Y., & Xing, Z. (2006). Robot path planning based on artificial potential field approach with simulated annealing. In *Sixth International Conference on Intelligent Systems Design and Applications* (Vol. 2, pp. 622–627). IEEE.
- [3] Zhang, H. Y., Lin, W. M., & Chen, A. X. (2018). Path planning for the mobile robot: A review. *Symmetry*, 10(10), 450.
- [4] Wang, H., Yu, Y., & Yuan, Q. (2011). Application of Dijkstra algorithm in robot path-planning. In *2011 Second International Conference on Mechanic Automation and Control Engineering* (pp. 1067–1069). IEEE.
- [5] Martinez-Alfaro, H., & Gomez-Garcia, S. (1998). Mobile robot path planning and tracking using simulated annealing and fuzzy logic control. *Expert Systems with Applications*, 15(3–4), 421–429.
- [6] Tsuzuki, M. D. S. G., de Castro Martins, T., & Takase, F. K. (2006). Robot path planning using simulated annealing. *IFAC Proceedings Volumes*, 39(3), 175–180.
- [7] Janabi-Sharifi, F., & Vinke, D. (1993). Integration of the artificial potential field approach with simulated annealing for robot path planning. In *Proceedings of 8th IEEE International Symposium on Intelligent Control* (pp. 536–541). IEEE.
- [8] Chen, H., Fuhlbrigge, T., & Li, X. (2009). A review of CAD-based robot path planning for spray painting. *Industrial Robot: An International Journal*, 36(1), 45–50.
- [9] Bedaka, A. K., & Lin, C. Y. (2018). CAD-based robot path planning and simulation using OPEN CASCADE. *Procedia Computer Science*, 133, 779–785.
- [10] Tavares, R. S., Martins, T. C., & Tsuzuki, M. D. S. G. (2011). Simulated annealing with adaptive neighborhood: A case study in off-line robot path planning. *Expert Systems with Applications*, 38(4), 2951–2965.